

Beyond the Black Box: Interpretability of Agentic AI Tool Use

Hariom Tatsat

Quantitative Analytics, Barclays
hariom.x.tatsat@barclays.com

Ariye Shater

Quantitative Analytics, Barclays
ariye.shater@barclays.com

Abstract

AI agents are promising for high-stakes enterprise workflows, but dependable deployment remains limited because tool-use failures are difficult to diagnose and control. Agents may skip required tool calls, invoke tools unnecessarily, or take actions whose consequence becomes visible only after execution. Existing observability methods are mostly external: prompts reveal correlations, and evaluations score outputs only after the model has already acted. In long-horizon settings, these failures are especially costly because an early tool mistake can alter the rest of the trajectory, increase token consumption, and create downstream safety and security risk.

We introduce a mechanistic-interpretability toolkit built on **Sparse Autoencoders (SAEs)** and **linear probes**. The framework reads model states before each action and infers both whether a tool is needed and how consequential the next tool action is likely to be. By decomposing activations into sparse features, it identifies the internal layers and features most associated with tool decisions and tests their functional importance through feature ablation. We train the probes on multi-step trajectories from the **NVIDIA Nemotron function-calling dataset** and apply the same workflow to **GPT-OSS 20B** and **Gemma 3 27B** models.

The goal is not to replace external evaluation, but to add a missing layer: visibility into what the model signaled internally before action. This helps surface deeper causes of agent failure, especially in long-horizon runs where an early mistake can reshape the rest of the agentic interaction. More broadly, the paper shows how mechanistic interpretability can support practical internal observability for monitoring tool calls and risk in agent systems.

1 Introduction

An *AI agent* solves a task through repeated decision steps rather than a single response. At each step, it can either answer directly or delegate to an external tool, observe the result, and continue. We study this *tool-decision boundary*: whether the model should call a tool now, and how consequential that action is likely to be.

This boundary is operationally important because tool-use failures are often difficult to diagnose before errors become visible. An agent may skip a required tool call, invoke a tool unnecessarily, or take an action whose consequence becomes visible only after execution. Standard observability methods remain incomplete here. Prompting reveals correlations rather than mechanisms, and behavioral evaluation measures outputs only after the model has already acted. The Berkeley Function Calling Leaderboard (BFCL) reflects this limitation in its multi-turn design by combining *state-based* and *response-based* checks, since read-only tool chains may be invisible to state-only evaluation (Patil et al., 2025). Related work on tool-selection hallucinations points in the same direction: hidden states can contain useful same-pass signals for tool-call errors that are not visible from outputs alone (Healy et al., 2026).

We address this gap with an internal monitoring framework that reads model activations immediately before each action and estimates whether the model is internally preparing to delegate to a tool and, secondarily, whether that action is likely to be low, medium, or high risk.

Mechanistic interpretability provides a mechanism to inspect model internals. Linear probes are classifiers trained on intermediate representations to test whether a target concept is linearly recoverable from them (Alain & Bengio, 2017). Sparse Autoencoders (SAEs) go further, decomposing activations into a sparse basis of more interpretable features (Bricken et al., 2023). We combine both: pre-action activations are first mapped into a sparse feature basis via an SAE, then read out by two task-specific probes: the **Tool-Need Probe** (binary: tool call vs. no tool call) and the **Tool-Risk Probe** (ternary: low / medium / high risk). This pipeline is lightweight yet interpretable, recovering tool-decision signals from internal state, pinpointing the layers where these signals are strongest, and surfacing the individual features most predictive of tool use and risk. Ablating those features then provides causal confirmation. The paper makes four contributions:

- A pre-action internal monitoring framework for repeated tool decisions in agent trajectories.
- Two complementary readouts: the Tool-Need Probe (Probe 1) and the Tool-Risk Probe (Probe 2).
- Localization of tool-decision signals to sparse features and late layers, with feature ablation.
- Evaluation on **GPT-OSS 20B** and **Gemma 3 27B instruction-tuned (IT)** models using held-out Nemotron test data and zero-shot BFCL transfer.

More broadly, the paper shows how mechanistic interpretability can support a practical form of internal observability for agent systems: not just explaining failures after the fact, but helping monitor tool calls and risk before execution.

Section 2 positions the paper relative to prior work and states the research questions. Section 3 defines the decision-point formulation, datasets, internal-state extraction, probe setup, and feature ablation method. Section 4 presents the main empirical results, including held-out Nemotron performance, the illustrative financial trace, layer concentration, and ablation. Section 5 evaluates held-out replay and zero-shot BFCL transfer, and Section 6 discusses implications, deployment considerations, and limitations.

2 Related Work and Research Questions

Prior work on tool use has mostly evaluated agents from the outside, through end-task success, function-call correctness, or benchmark-specific response scoring. This external evaluation tradition includes learned tool-use setups such as Toolformer and broader function-calling / API benchmarks such as ToolLLM / ToolBench, ToolACE, HammerBench, and BFCL (Schick et al., 2023; Qin et al., 2023; Liu et al., 2024; Wang et al., 2025; Patil et al., 2025). BFCL is the most relevant benchmark for our setting because it evaluates abstention and multi-turn behavior, and combines *state-based* and *response-based* checks when read-only tool chains are not visible from final state alone. These benchmarks are essential for measuring observable behavior, but they do not reveal whether the model had internally recognized the need to delegate before acting.

A second line of work studies hidden states directly. Activation-probing results show that internal representations can predict downstream behavior before it is externally visible (Li et al., 2025; McKenzie et al., 2025). In tool-selection settings, internal representations have also been shown to distinguish correct from hallucinated tool calls, with calibration playing an important role when such probes are intended for deployment rather than only offline analysis (Healy et al., 2026). In parallel, Sparse Autoencoder work shows that dense activations can be decomposed into more interpretable sparse features, making it possible to localize semantically meaningful internal components rather than operate only on opaque residual vectors (Bricken et al., 2023; Cho et al., 2025).

Our work connects these directions in a multi-step agent setting. Rather than evaluating tool use only from outputs, we monitor model state immediately before each action. Rather than probing dense hidden states alone, we probe SAE features that support layer localization, sparse feature inspection, and ablation. This is especially relevant for long-horizon agents, where early tool or coordination failures can propagate through the rest of the trajectory (Cemri et al., 2025). It is also aligned with the view that external tools should be invoked when they are epistemically necessary, rather than reflexively (Wang et al., 2025). It also complements our earlier finance-focused study of mechanistic

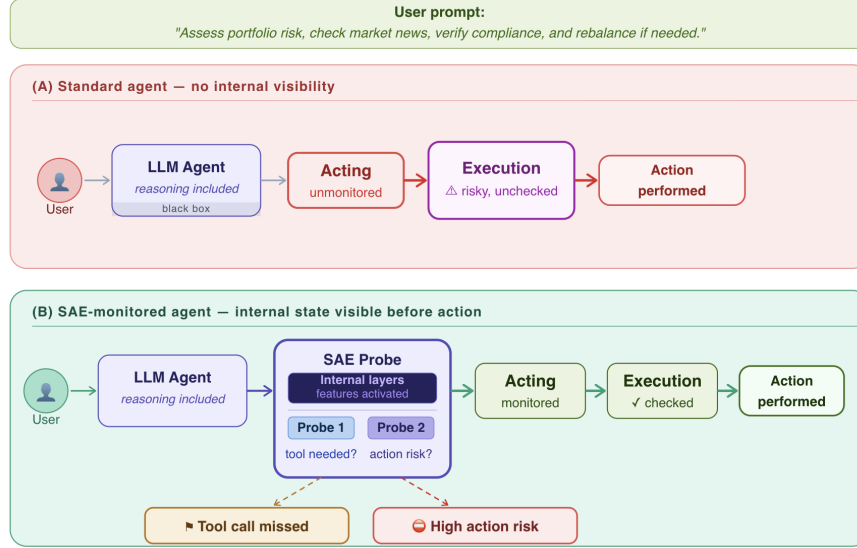


Figure 1: Framework overview for mechanistic monitoring of multi-step agent tool decisions. Agent trajectories are transformed into decision-boundary contexts, mapped into pre-action activations, decomposed with Sparse Autoencoders, and used by the Tool-Need Probe (Probe 1) and Tool-Risk Probe (Probe 2) before execution.

Table 1: Operational outcomes used throughout the paper.

Outcome	Definition
Correct no-tool	No tool is required, and no tool is used.
Correct tool use	A tool is required, and the runtime calls a tool.
Missed tool call	A tool is required, but the runtime does not call one.
Unnecessary tool call	No tool is required, but the runtime calls one anyway.
High-risk tool call	A tool is used and Probe 2 assigns a high-risk tier.
Uncertain decision	The internal tool signal is weak or unstable near the decision boundary.

interpretability, which examined domain-specific LLM behavior rather than pre-action tool decisions in agent trajectories (Tatsat & Shater, 2025).

The paper is organized around four research questions. **RQ1** asks whether model activations encode whether a tool should be used at a given decision step. **RQ2** asks which sparse features and layers most strongly encode tool-need and tool-risk signals. **RQ3** asks whether internal signals can surface missed and unnecessary tool calls more clearly than output-only monitoring. **RQ4** asks whether these signals remain useful across repeated decision points and under zero-shot transfer to BFCL. RQ1 and RQ2 are addressed mainly in Sections 3 and 4; RQ3 and RQ4 are addressed mainly in Sections 4, 5, and 6.

3 Problem Setup and Method

We study agent behavior at repeated *tool decision points*. At each step, we compare three quantities: what the task requires, what the model internally signals, and what the runtime actually does. This three-way view lets us distinguish the main cases that matter for monitoring: correct tool use, missed tool calls, unnecessary tool calls, high-risk actions, and uncertain decisions. Figure 1 summarizes the full decision-boundary pipeline.

Table 1 summarizes the operational outcomes used throughout the paper. The Tool-Need Probe provides the internal tool signal, while the Tool-Risk Probe estimates the likely risk tier of the next tool action. This compact framing replaces a longer failure taxonomy while preserving the cases that matter most for runtime monitoring.

3.1 Data preparation

We convert raw multi-step agent trajectories into per-step decision rows. Each row contains cumulative context truncated at the decision boundary, a binary label indicating whether a tool is required, and a three-level risk label for the next tool action. This preserves a faithful pre-action view: the probe never sees the current step’s output or the future trajectory when computing its prediction.

The training data comes from the NVIDIA Nemotron function-calling dataset (Chandiramani et al., 2026), where each raw row corresponds to one decision point in a multi-step trajectory. We group rows by trajectory, order them by depth, reconstruct the cumulative context available at each step, and assign a binary `tool_needed` label from the gold next action. Tool-call steps are additionally assigned one of three risk tiers: **low**, **medium**, or **high**. Low-risk actions are predominantly read-only retrieval or lookup steps; medium-risk actions involve bounded creation or write operations; high-risk actions include authentication, outbound communication, or dangerous execution actions. Table 13 in Appendix B summarizes the keyword groups used to instantiate this Nemotron risk-tier scheme.

Probes are trained only on Nemotron-derived step rows. BFCL is reserved for zero-shot transfer evaluation, using the same per-step reconstruction and pre-action probe inference but a different benchmark distribution.

3.2 Internal state extraction

We apply the same decision-point pipeline to both backbones: identical per-step context, omission of the current step’s generated output from the activation prompt, and layer-wise SAE encoding of pre-action hidden states. For both models, hidden states are mean-pooled over the last 32 pre-action tokens before SAE encoding, rather than read from a single token alone. This choice provides a practical balance between capturing enough immediate context to stabilize the decision signal and keeping activation extraction computationally manageable at runtime.

For **GPT-OSS 20B**, we read six post-residual layers and encode them with public GPT-OSS SAEs. For **Gemma 3 27B**, we read four post-block residual layers and encode them with Gemma Scope SAEs. The important point for the paper is not the exact dimensionality of each concatenated vector, but that both models are processed with the same decision-boundary logic and the same probe-based monitoring recipe.

3.3 Probe training

The **Tool-Need Probe** is the primary probe: it predicts whether a tool call is required at the current decision step. The **Tool-Risk Probe** is secondary: at tool-call steps it predicts whether the next action is low, medium, or high risk. Both probes operate on SAE features rather than raw activations, which makes it possible to inspect layer concentration, identify top sparse features, and test feature necessity through ablation.

Formally, let $\tilde{h}^{(\ell)} \in \mathbb{R}^d$ denote the pooled pre-action hidden state at layer ℓ . For each selected layer, a pretrained SAE maps this hidden state to a sparse feature vector

$$z^{(\ell)} = \phi\left(W_{\text{enc}}^{(\ell)}\tilde{h}^{(\ell)} + b_{\text{enc}}^{(\ell)}\right),$$

where $W_{\text{enc}}^{(\ell)}$ and $b_{\text{enc}}^{(\ell)}$ are the SAE encoder weights and bias for layer ℓ , and $\phi(\cdot)$ denotes the SAE nonlinearity. We concatenate SAE features across the selected layers,

$$z = [z^{(\ell_1)}; \dots; z^{(\ell_m)}],$$

where m is the number of selected layers. We then fit linear probes on z rather than on raw activations. For Tool-Need, with binary label $y \in \{0, 1\}$,

$$p(y = 1 \mid z) = \sigma(w^\top z + b),$$

where w and b are probe parameters and $\sigma(\cdot)$ is the logistic sigmoid. Tool-Risk uses a three-way softmax over $\{\text{low}, \text{med}, \text{high}\}$. The two probes are trained independently, with distinct targets and feature-ranking criteria, but are evaluated under the same per-step runtime framework. All remaining notation follows standard linear-algebra conventions.

Table 2: Main results: GPT-OSS 20B vs Gemma 3 27B instruction-tuned (IT) on the core held-out Nemotron test dimensions.

Metric	GPT-OSS 20B	Gemma 3 27B
<i>Held-out Nemotron test</i>		
Tool-Need accuracy	75.3%	71.4%
Tool-Need F1 (macro)	0.75	0.71
Tool-Risk accuracy (tool rows)	90.3%	88.5%
Tool-Risk macro-F1	0.64	0.62
<i>Layer concentration</i>		
Dominant SAE layers	23, 19	40, 53, 31

Each probe is implemented as a sparse logistic classifier over SAE features, with feature selection based on how well each feature separates the target classes and regularization chosen from ridge, lasso, or elastic net. Regularization is applied because the SAE feature space is high-dimensional and often contains correlated latents, so some shrinkage helps control overfitting while keeping the readout interpretable.

To make representative SAE features easier to interpret, we apply an automated feature-labeling step to a small number of selected features. In this workflow, top-activating examples are summarized into short natural-language descriptions using a LLM. More details are deferred to Appendix A.

3.4 Feature ranking and ablation

To test whether top sparse features are merely correlated with probe predictions or are functionally necessary, we perform representational ablation directly in SAE latent space. After encoding a step into sparse features, we select a small set of highly-ranked latents, zero them, re-run the probe, and compare the ablated prediction with the baseline prediction. If suppressing a small set of latents sharply reduces probe confidence or flips the label, those features are causally important to the probe’s prediction.

3.5 Evaluation metrics

We report Tool-Need accuracy, precision, recall, and F1; Tool-Risk accuracy and macro-F1; and runtime alignment between expected labels, internal probe decisions, and actual execution. We also report missed-tool warning rates, unnecessary-call warning rates, and risk alerts in replay and transfer settings.

4 Experiments and Results

Table 2 gives the headline held-out Nemotron results across both models. The key result is that the Tool-Need Probe (Probe 1) provides the stronger and more stable signal at the tool-call boundary, while the Tool-Risk Probe (Probe 2) becomes useful once a tool call is warranted, where it helps distinguish lower from higher risk actions but is more sensitive to risk-class structure and transfer setting. In both models, the strongest tool-decision information is concentrated in later layers. As described in Section 3 under *Probe training*, the number of features varies across probes: GPT-OSS Probe 1 uses lasso with 200 features, all other probes use elastic net, Gemma Probe 1 uses 2000 features, and Probe 2 uses 1000 for both models.

4.1 Tool-Need Probe results

Both models contain a linearly recoverable tool-decision signal in their SAE representations.

GPT-OSS achieves **75.3%** accuracy on the held-out Nemotron test, while Gemma achieves **71.4%**. The confusion matrices in Table 3 show that both probes behave as reasonably balanced classifiers rather than simply exploiting label frequency. This tool-need signal is the paper’s primary contribution: it is recoverable with compact feature sets, interpretable in layer space, and later transfers as an omission-auditing signal in runtime settings.

Table 3: Tool-Need Probe confusion matrices on held-out Nemotron test for GPT-OSS and Gemma.

GPT-OSS (1,993 rows)		
	Pred: 0	Pred: 1
True: no_tool	741	225
True: tool	267	760

Gemma (1,821 rows)		
	Pred: 0	Pred: 1
True: no_tool	610	284
True: tool	236	691

Table 4: Tool-Risk Probe confusion matrices on held-out Nemotron data. Rows = true tier; columns = predicted tier.

GPT-OSS (987 rows)			
	Low	Med	High
True: low	818	40	18
True: med	16	24	2
True: high	17	3	49

Gemma (1,004 rows)			
	Low	Med	High
True: low	807	35	28
True: med	22	21	2
True: high	21	7	61

Table 5: Illustrative steps from the Nemotron financial trajectory (trajectory_id 3344). Probe 2 probabilities are shown as $(p_{\text{low}}, p_{\text{med}}, p_{\text{high}})$.

Step	Summarized prompt	p_{tool}	Probe 2 (L, M, H)
4	Upcoming IPO listings	0.846	(0.992, 0.005, 0.003)
7	J&J earnings schedule	0.632	(0.996, 0.003, 0.002)
10	Brookdale balance sheet	0.548	(0.993, 0.004, 0.004)
12	TTD balance sheet / liquidity	0.656	(0.988, 0.012, 0.000)
14	Tesla income statement	0.881	(0.997, 0.003, 0.000)
15	Follow-up after Tesla request	0.202	(0.438, 0.484, 0.078)

Table 6: Layer concentration of top-20 Probe-1 features by model.

GPT-OSS	L3	L7	L11	L15	L19	L23
	0	0	2	1	4	13

Gemma	L16	L31	L40	L53
	0	5	7	7

4.2 Tool-Risk Probe results

Knowing that a tool should be called is necessary but not sufficient. Tool actions can differ substantially in risk even when both are valid. Probe 2 therefore asks whether internal representations encode not only the decision to call a tool, but the severity of the next external action.

Both models show strong held-out Nemotron accuracy: **90.3%** for GPT-OSS and **88.5%** for Gemma on tool-call rows. The dominant pattern is strong low-tier recovery, weaker medium-tier separation, and moderate high-tier recovery. This asymmetry is consistent across both models, suggesting it reflects the risk-tier structure rather than a model-specific artifact. Table 4 makes this pattern explicit.

4.3 Illustrative multi-step financial information trace

We retain one illustrative financial trajectory from the Nemotron distribution (trajectory_id 3344) to show how the two probes evolve across repeated decision points. This subsection is intended as a worked trace rather than as a standalone evaluation. The quantitative evidence for the paper remains the held-out metrics, confusion matrices, feature tables, and ablation results; the full step-by-step trace and the corresponding plot appear in Appendix C, specifically Figure 2 and Table 15.

The main value of this example is that the **Tool-Need Probe (Probe 1)** rises on steps that genuinely require external financial retrieval and falls on follow-up turns where no new tool call is needed, even though the discussion remains financial. The **Tool-Risk Probe (Probe 2)** stays predominantly low on those retrieval steps, which shows that the risk signal is not redundant with tool need.

Table 5 summarizes the key steps, abbreviated prompts, and probe outputs.

The key transitions are intuitive. Steps 4, 7, 10, 12, and 14 are read-only financial-information retrieval requests, and these steps coincide with elevated Probe 1 scores while Probe 2 remains predominantly low. Step 15 differs: it is a follow-up turn in the same conversation, and here p_{tool} drops while the Probe 2 distribution becomes more mixed. Taken on its own, this trace is only

Table 7: Representative top SAE features for Probe 1 (Tool-Need), shown separately for GPT-OSS and Gemma.

GPT-OSS (top 5)		
Layer	Feature	Label
23	79,265	mathematical expressions, numbers
23	90,074	numbers and numerical data
23	106,054	legal and formal language
23	106,420	numerical values, sequences
23	38,964	coordinates, measurements

Gemma (top 5)		
Layer	Feature	Label
53	1,694	punctuation, numbers, list sequences
40	204	numerical data: quantities, years
40	1,084	high-freq common words, punctuation
53	2,322	professional training, evaluative terms
40	166	nouns: projects, systems, structures

Table 8: Representative top SAE features for Probe 2 (Tool-Risk), shown separately for GPT-OSS and Gemma.

GPT-OSS		
Layer	Feature	Label
23	63,701	password generation / programming
23	58,277	notable figures, formal declarations
19	80,633	motivation / overcoming challenges
23	106,054	legal / formal policy language
23	38,964	coordinates, measurements, data

Gemma		
Layer	Feature	Label
53	6,032	usernames, passwords, authentication
53	10,969	password creation, security management
53	3,246	logging in, accounts, authentication
53	21,923	password formats and examples
40	10,969	password / account security

Table 9: Tool-Need Probe ablation results (10 held-out Nemotron steps per model). Δp = mean $|\Delta p_{\text{tool}}|$; Flip = binary prediction flips out of 10.

Model	Set	# Latents	Flips	Mean $ \Delta p $
GPT-OSS	Top-5	5	3/10	0.236
GPT-OSS	Top-10	10	4/10	0.431
GPT-OSS	Top-20	20	4/10	0.384
Gemma	Top-50	50	0/10	0.031
Gemma	Top-100	100	0/10	0.058
Gemma	Top-200	200	1/10	0.146

Table 10: Held-out Nemotron replay runtime profile (760 episodes, GPT-OSS).

Metric	Value
Step accuracy	78.6%
Missed-tool-call rate	34.2% of tool-required steps
Unnecessary-call rate	7.7% of no-tool-required steps
Tool-naming accuracy (given call)	90.8%
Probe catch (missed tool)	~75.5% (~258/343)
Fully correct episodes	62.5% (475/760)
Mean first-failure step	0.72 turns
Mean progress before failure	66.6%

illustrative, but it helps show how the probe outputs can evolve differently across successive decision points within a single multi-step interaction.

4.4 Layer and feature analysis

Both models concentrate their tool-decision signal in late transformer layers. Table 6 shows how the top Probe 1 features cluster toward the final monitored layers in both backbones.

This pattern is consistent with a late-stage decision-encoding hypothesis: the strongest tool-decision features appear near the model’s final commitment stage before action.

Table 7 reports representative top SAE features for Probe 1 (Tool-Need), and Table 8 reports the corresponding representative features for Probe 2 (Tool-Risk).

These feature tables matter because they show that the probes are not only accurate but also readable at the feature level. Table 7 highlights the numerical and formal-language features associated with tool-call decisions, while Table 8 shows that the strongest Probe 2 features emphasize authentication, account, and credential-related concepts rather than tool names alone. This supports the claim that Probe 2 reads action consequence from reasoning context rather than from a static tool-name lookup.

4.5 Feature ablation

Representational ablation confirms that the top sparse features are functionally necessary, not merely correlated. Table 9 summarizes the effect of ablating high-ranked latent sets.

For GPT-OSS, a small set of top-ranked features is sufficient to sharply reduce p_{tool} and flip several predictions, with most of the strongest ablated features coming from the final layer. Gemma shows the same pattern more gradually, requiring larger ablation sets because the probe spans a broader

Table 11: BFCL zero-shot transfer summary

Metric	GPT-OSS 20B	Gemma 3 27B
<i>Runtime performance</i>		
Expected–Actual agreement	83.6%	87.6%
Missed-tool rate (of tool steps)	10.8%	0.2%
Unnecessary-call rate (of no-tool steps)	56.3%	98.5%
Episode success (all steps correct)	49.2%	50.0%
Mean first-failure step	1.69 turns	1.17 turns
<i>Probe quality</i>		
Probe-1 agreement with gold (E vs I)	87.7%	77.7%

sparse input. Random-feature ablation produces negligible effects, supporting the claim that the identified features are specific components of the signal the probe reads out.

5 Runtime Monitoring and Cross-Dataset Generalization

5.1 Training and transfer setup

Probes are trained on **Nemotron** data only. Held-out **Nemotron replay** provides the same-distribution runtime check for GPT-OSS 20B, while **BFCL** is used as a strict zero-shot transfer benchmark with no retraining or threshold tuning (Patil et al., 2025). This section therefore separates familiar-distribution replay from cross-benchmark transfer.

5.2 Held-out Nemotron replay

We first evaluate the monitor on held-out Nemotron replay using GPT-OSS 20B, the same model family used for activation extraction. The main result is that missed tool calls dominate, while tool naming is comparatively strong once the model commits to a tool call. This supports interpreting Probe 1 primarily as a *tool-call monitor* rather than a formatting checker. Table 10 summarizes the replay profile.

Held-out Nemotron replay shows that the central problem is not naming a tool once the model has decided to use one; it is deciding to delegate at all. This is the clearest runtime justification for Probe 1.

5.3 BFCL as a zero-shot transfer benchmark

BFCL is used here as a strict zero-shot transfer benchmark and as the clearest cross-benchmark generalization test. No BFCL activations are used for training, retraining, threshold tuning, or calibration. Instead, BFCL multi-turn episodes are mapped into the same step-level format used for Nemotron: cumulative transcript becomes context, gold call annotations determine `tool_needed`, and BFCL tools are heuristically projected into the Nemotron low/medium/high risk-tier scheme for Probe 2. This preserves the same pre-action monitoring setup while exposing the probes to a different benchmark distribution. Table 11 summarizes the transfer results, while Appendix C includes Table 14, an illustrative formatting contrast between the two benchmark styles.

The main out-of-distribution result is tool-call transfer. Both models show expected–internal agreement above 77% on BFCL despite no BFCL activations being used in training, indicating transfer beyond Nemotron. GPT-OSS Probe 1 predicts a tool on nearly every BFCL step, while Gemma retains two-sided behavior with a 0.2% missed-tool rate but still over-triggers on no-tool steps. Probe 1 therefore transfers usefully to BFCL, though the two models show different error profiles under distribution shift.

Probe 2 also shows useful but a slightly weaker transfer. This is not surprising, since cross-benchmark risk transfer depends on how well tool categories and risk tiers are specified and aligned across datasets.

Failures are early in both models: mean first-failure occurs within the first one or two turns, including 1.17 turns for Gemma on the merged BFCL slice. This suggests a short but practically relevant intervention window for pre-execution monitoring. Overall, BFCL should be read as a transfer stress

test: Probe 1 transfers best as an omission-auditing signal at the tool-call boundary, while Probe 2 remains useful for separating lower- from higher-risk actions once a tool call is in play.

6 Discussion and Limitations

The broader contribution is to show that mechanistic interpretability can become **operationally useful** for agent systems: extending our earlier *Beyond the Black Box* study of LLM interpretability (Tatsat & Shater, 2025) into agent settings, this paper shows how internal monitoring can move beyond explaining behavior after failure to helping monitor tool decisions before action in realistic high-stakes workflows.

A second advantage is generality. Because the monitor operates on internal representations rather than tool-specific output patterns, the same probe framework can apply across multiple tools and repeated decision points. Probe 1 asks whether a tool call is needed at all, while Probe 2 asks whether the next tool action appears more consequential; feature tables and ablations show that these signals are both predictive and localized to late sparse features.

This is also where SAE-based monitoring adds something beyond external observability. The framework does not simply emit a scalar warning; it identifies where the signal is concentrated, which sparse features are most associated with the decision, and whether suppressing those features changes the probe output.

The runtime results suggest that the framework is most useful as an oversight layer at high-value decision points. Held-out Nemotron replay shows that the main bottleneck is deciding to delegate at all, not naming a tool once delegation has begun. BFCL then serves as a transfer stress test: Probe 1 transfers best as an omission-auditing signal, while Probe 2 remains useful for risk-tiering once a tool call is underway. We therefore interpret the Nemotron–BFCL gap as reflecting both task transfer and added benchmark-specific instruction-following pressure, not simply loss of the internal signal itself.

Limitations. Tool-Need is the stronger and more stable probe, while Tool-Risk depends more directly on the risk-tier scheme used to label actions. The Nemotron and BFCL risk labels are heuristic rather than a shared industry standard, so out-of-domain risk predictions should be interpreted with caution. More broadly, BFCL is not only a distribution shift in tool tasks; it also changes the amount of benchmark-specific instruction-following needed for a model to be scored as correct. That makes BFCL an intentionally demanding transfer setting, but also means that BFCL degradation can reflect a mixture of capability shift, protocol mismatch, and judge-facing compliance effects. The paper also studies only two open-weight backbones, so broader portability across architectures, scales, and post-training recipes remains an open question. Finally, feature identities may drift with checkpoint choice and SAE recipe, even when late-layer concentration appears robust. Additional examples and per-step tables appear in Appendix C.

7 Conclusion

Tool decisions leave readable traces in model state *before* external execution. On Nemotron-held trajectories, linear probes on SAE-decomposed activations recover Tool-Need and Tool-Risk signals for both **GPT-OSS 20B** and **Gemma 3 27B IT**, with late-layer concentration and sparse feature sets that survive ablation: evidence that the readout targets a genuine internal signal, not an arbitrary projection of the residual stream.

The same monitoring recipe transfers across backbones under distinct SAE variants and layer selections and remains informative under live replay and BFCL out-of-distribution evaluation, with Tool-Need acting most reliably as an **omission auditor** and Tool-Risk as a **risk-oriented** layer that requires extra care when the risk scheme shifts across tool namespaces. Selective activation capture at decision boundaries therefore offers a practical complement to external benchmarks for safer, more controllable agent deployment.

The broader contribution is to show that mechanistic interpretability can become **operationally useful** for agent systems: this paper shows how internal monitoring can move beyond explaining behavior after failure to helping monitor tool decisions before action in realistic high-stakes workflows.

References

- Alain, G., & Bengio, Y. (2017). Understanding Intermediate Layers Using Linear Classifier Probes. *arXiv preprint arXiv:1610.01644*. <https://arxiv.org/abs/1610.01644>
- Bricken, T., et al. (2023). Towards Monosemanticity: Decomposing Language Models with Dictionary Learning. *Transformer Circuits Thread*. <https://transformer-circuits.pub/2023/monosemantic-features/>
- Cemri, M., et al. (2025). Why Do Multi-Agent LLM Systems Fail? *arXiv preprint arXiv:2503.13657*. <https://arxiv.org/abs/2503.13657>
- Chandiramani, A., et al. (2026). Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. *arXiv preprint arXiv:2604.12374*. <https://arxiv.org/abs/2604.12374>
- Cho, S., Wu, Z., & Koshiyama, A. (2025). CorrSteer: Steering Improves Task Performance and Safety in LLMs through Correlation-based Sparse Autoencoder Feature Selection. *arXiv preprint arXiv:2508.12535*. <https://arxiv.org/abs/2508.12535>
- Healy, K., et al. (2026). Internal Representations as Indicators of Hallucinations in Agent Tool Selection. *arXiv preprint arXiv:2601.05214*. <https://arxiv.org/abs/2601.05214>
- Li, W., et al. (2025). Adaptive Tool Use in Large Language Models with Meta-Cognition Trigger. *arXiv preprint arXiv:2502.12961*. <https://arxiv.org/abs/2502.12961>
- Liu, W., et al. (2024). ToolACE: Winning the Points of LLM Function Calling. *arXiv preprint arXiv:2409.00920*. <https://arxiv.org/abs/2409.00920>
- McKenzie, A., et al. (2025). Detecting High-Stakes Interactions with Activation Probes. *arXiv preprint arXiv:2506.10805*. <https://arxiv.org/abs/2506.10805>
- Patil, S. G., et al. (2025). The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. *Proceedings of the Forty-Second International Conference on Machine Learning*. <https://openreview.net/forum?id=2GmDdhBdDk>
- Qin, Y., et al. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. *arXiv preprint arXiv:2307.16789*. <https://arxiv.org/abs/2307.16789>
- Rai, D., et al. (2024). A Practical Review of Mechanistic Interpretability for Transformer-Based Language Models. *arXiv preprint arXiv:2407.02646*. <https://arxiv.org/abs/2407.02646>
- Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*, 36. <https://arxiv.org/abs/2302.04761>
- Tatsat, H., & Shater, A. (2025). Beyond the Black Box: Interpretability of LLMs in Finance. *arXiv preprint arXiv:2505.24650*. <https://arxiv.org/abs/2505.24650>
- Wang, J., et al. (2025). HammerBench: Fine-Grained Function-Calling Evaluation in Real Mobile Device Scenarios. *arXiv preprint arXiv:2412.16516*. <https://arxiv.org/abs/2412.16516>
- Wang, H., et al. (2025). Position: Agent Should Invoke External Tools ONLY When Epistemically Necessary. *arXiv preprint arXiv:2506.00886*. <https://arxiv.org/abs/2506.00886>

A FEATURE LABELING METHODOLOGY

Table 12 summarizes the paper’s *feature labeling methodology* for representative SAE features. We use it only to provide a small number of compact feature labels that help the reader interpret the probe tables.

For each selected feature, we retrieve top-activating examples, convert them into a short evidence summary, and use that evidence to produce a concise candidate label. When available, we also use “Neuronpedia” as an external automated-interpretability browser for inspecting SAE features and activation examples; its descriptions are treated only as a cross-check rather than as ground truth. The resulting labels are therefore best understood as brief interpretability anchors, not as definitive names for model mechanisms.

Table 12: Technical summary of the feature labeling methodology used for representative SAE features.

Stage	Object	Technical role
Feature specification	model ID + SAE source + feature index	Defines the exact latent to be labeled.
Activation retrieval	top-activating examples	Collects the strongest evidence for that feature from Neuronpedia, local inference, or precomputed activations.
Evidence formatting	token/value pairs or top text spans	Converts raw activation evidence into a compact explanation prompt.
Label generation	local or API-based labeling model	Produces a concise candidate description for the selected feature.
Cross-checking	candidate label + retrieved evidence	Checks that the label matches the strongest activation contexts.
Output	short feature label	Reports a compact interpretability anchor for representative features in the paper.

B NEMOTRON RISK-TIER SCHEME

This appendix summarizes the keyword groups used to instantiate the Nemotron risk-tier scheme referenced in Section 3.1. The scheme is heuristic and is intended to capture the operational risk of the next tool action rather than its domain label. Table 13 lists the tier definitions and representative keyword groups.

Table 13: Keyword groups used in the Nemotron risk-tier scheme. Representative tools are illustrative rather than exhaustive.

Tier	Distinct tools	Operational meaning	Representative keyword groups / tools
Low	920	Read-only retrieval, lookup, and search operations with no persistent state change	Search, retrieval, lookup, financial data access, read-only inspection
Medium	27	Bounded write or creation operations with contained scope	write_file, text_editor, create_desktop_txt_file, run-code, generateImageUrl, image_generation, generateImage, create_room, create_subdirectory, save_as_pdf
High	21	Authentication, outbound communication, or dangerous execution actions with external consequence or irreversibility	getusertoken, registeruser, modifypassword, forgotpassword, deleteaccount, sendemail, sendim, sendmessage, shell-exec, execute_command, python_exec, exec, execute_bash_code, run_zapier_NLA_action

C ADDITIONAL FINANCIAL QUALITATIVE EXAMPLES

This appendix collects the supplementary qualitative traces that support the worked example in Section 4.3. Figure 2 covers the multi-ticker fundamentals trace, Figures 3 and 4 cover the Bitcoin DCA trace, Table 14 provides the illustrative Nemotron–BFCL formatting contrast referenced in Section 5, and Tables 15–17 provide the corresponding full per-step values.

C.1 Illustrative cross-benchmark formatting example

Table 14 shows the non-evaluative formatting contrast referenced in Section 5. It is included only to illustrate how similar tool-use logic can appear under different benchmark representations.

Table 14: Illustrative formatting contrast between Nemotron and BFCL. The table is included only to show how similar tool-use logic can appear under different benchmark representations; it is not itself an evaluation result.

Dataset	Illustrative prompt format
Nemotron (trajectory-style)	User: “I have \$450, spent \$150 on groceries and \$50 on utilities. How much remains, and what is 20% for emergency savings?” Reasoning trace: compute remaining amount, then compute 20%. Tool calls: <code>subtract({a:450,b:150}) → 300; subtract({a:300,b:50}) → 250.</code> Assistant: “You have \$250 left; save \$50.”
BFCL (benchmark-style)	question: “Read finance values and compute remaining cash; then compute 20% savings.” ground_truth: <code>[subtract(a=450,b=150), subtract(a=300,b=50), echo(content='50',file_name='savings.txt')]</code>

C.2 Multi-ticker fundamentals trace (trajectory_id 3344)

Figure 2 shows the Tool-Need probability curve for the multi-ticker fundamentals trace discussed in Section 4.3; the full step-level values appear in Table 15.

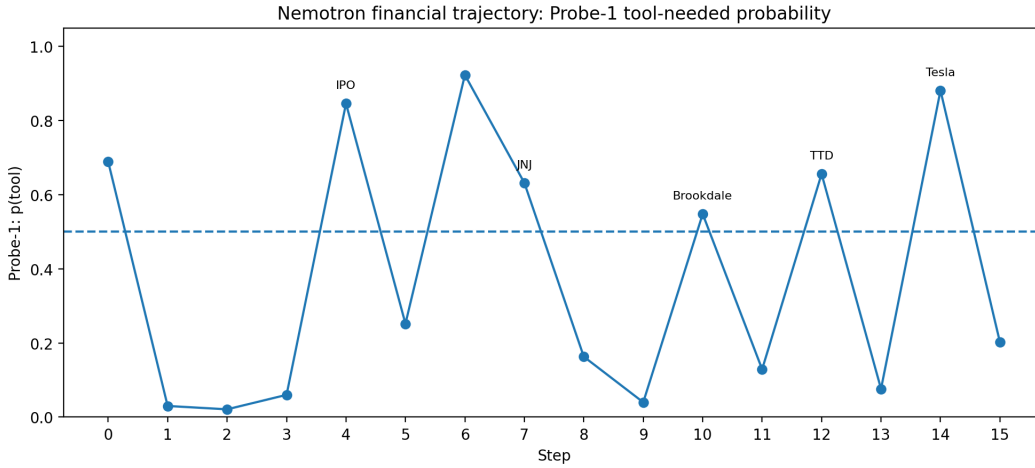


Figure 2: Tool-Need Probe (Probe 1) on the multi-ticker fundamentals trajectory. The signal rises on steps that require external financial retrieval and falls on follow-up no-tool steps.

C.3 Bitcoin DCA scenario (Nemotron, trajectory_id 4592)

Figures 3 and 4 show the corresponding Tool-Need and Tool-Risk traces for the Bitcoin DCA scenario; the full step-level values appear in Table 16.

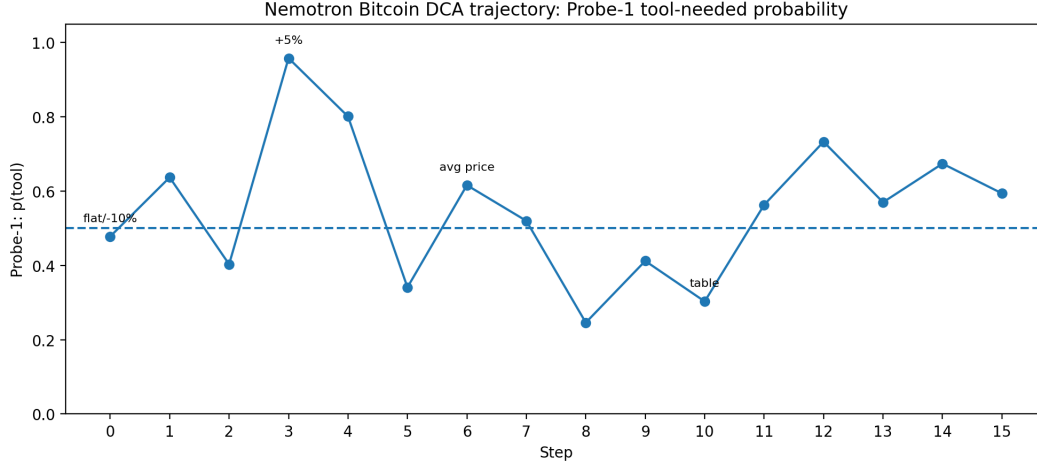


Figure 3: Tool-Need Probe (Probe 1) on the Bitcoin DCA trajectory. The tool-needed signal rises on calculation-heavy steps and falls on intervening no-tool steps.

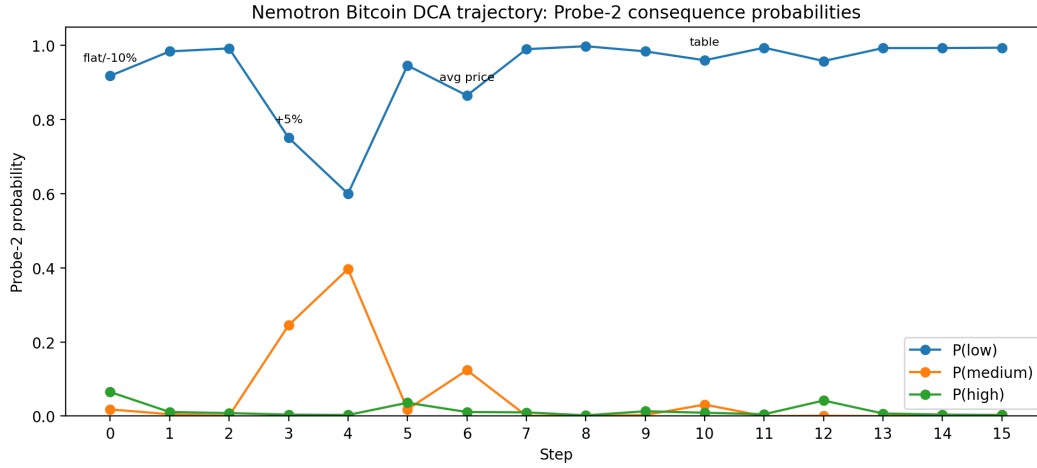


Figure 4: Tool-Risk Probe (Probe 2) on the Bitcoin DCA trajectory. Risk probabilities remain overwhelmingly low, consistent with calculator-style actions.

C.4 Full per-step probe tables

Tables 15, 16, and 17 report the full step-level outputs underlying the qualitative examples and BFCL slice referenced in the main text.

Table 15: Nemotron multi-ticker fundamentals (trajectory_id 3344): all pivot steps (GPT-OSS probes). “Phase” summarizes the latest user intent at each step; tier = Probe-2 argmax.

Step	Phase / user intent	p_{tool}	Tier	p_{low}	p_{med}	p_{high}
0	Confirm SU + API key after Suncor ask	0.689	low	0.999	0.000	0.001
1	Same turn (extended context)	0.030	low	0.886	0.019	0.094
2	Emerson EMR: income & cash flow	0.021	low	0.950	0.031	0.020
3	IBM: overview & ratios	0.060	low	0.990	0.007	0.003
4	IPO names / listings	0.846	low	0.992	0.005	0.003
5	Same IPO thread	0.252	low	0.985	0.001	0.015
6	Pivot: healthcare / senior living	0.923	low	0.936	0.049	0.016
7	JNJ: earnings schedule (12 months)	0.632	low	0.996	0.003	0.002
8	Same JNJ request	0.163	low	0.939	0.033	0.028
9	Same JNJ request	0.040	low	0.881	0.001	0.118
10	Brookdale: balance sheet	0.548	low	0.993	0.004	0.004
11	Same Brookdale thread	0.129	low	0.988	0.003	0.009
12	The Trade Desk (TTD): balance sheet / liquidity	0.656	low	0.988	0.012	0.000
13	Same TTD thread	0.076	low	0.868	0.006	0.126
14	Tesla: income statement & revenue trends	0.881	low	0.997	0.003	0.000
15	Same Tesla thread	0.202	medium	0.438	0.484	0.078

Table 16: Nemotron Bitcoin DCA scenario (trajectory_id 4592): all pivot steps (GPT-OSS probes).

Step	Phase / user intent	p_{tool}	Tier	p_{low}	p_{med}	p_{high}
0	Opening DCA question (flat vs. 10%/mo decline)	0.478	low	0.918	0.018	0.065
1	Same opening (more tool traces in context)	0.637	low	0.984	0.005	0.011
2	Same opening	0.403	low	0.992	0.001	0.008
3	Add +5%/month price path	0.958	low	0.751	0.245	0.004
4	Same follow-up	0.801	low	0.600	0.397	0.003
5	Same follow-up	0.341	low	0.946	0.018	0.036
6	Average purchase price per BTC, three scenarios	0.616	low	0.865	0.124	0.011
7	Same	0.520	low	0.990	0.001	0.010
8	Same	0.246	low	0.998	0.000	0.002
9	Same	0.412	low	0.984	0.003	0.013
10	Tabular summary: avg price & totals	0.303	low	0.960	0.031	0.009
11	Same table request	0.563	low	0.994	0.000	0.005
12	Same	0.733	low	0.958	0.001	0.042
13	Same	0.570	low	0.993	0.000	0.007
14	Same	0.674	low	0.993	0.003	0.004
15	Same	0.594	low	0.994	0.003	0.003

Table 17: BFCL trading episode (multi_turn_base_102): all steps in the evaluated slice (GPT-OSS probes). Gold risk = heuristic projection used for expected tier.

Step	p_{tool}	Tier	p_{low}	p_{med}	p_{high}	Gold risk	Expected tool
0	0.997	high	0.014	0.002	0.984	high	place_order
1	0.957	high	0.327	0.005	0.668	low	get_order_details
2	0.969	high	0.175	0.013	0.812	medium	cancel_order
3	0.767	high	0.160	0.003	0.837	low	get_account_info
4	0.820	high	0.017	0.000	0.983	medium	create_ticket